

Planner (often seen in publications as "PLANNER" although it is not an acronym) is a programming language designed by Carl Hewitt at MIT, and first published in 1969. First, subsets such as Micro-Planner and Pico-Planner were implemented, and then essentially the whole language was implemented as Popler by Julian Davies at the University of Edinburgh in the POP-2 programming language. Derivations such as QA4, Conniver, QLISP and Ether (see scientific community metaphor) were important tools in artificial intelligence research in the 1970s, which influenced commercial developments such as Knowledge Engineering Environment (KEE) and Automated Reasoning Tool (ART).

Procedural approach versus logical approach

The two major paradigms for constructing semantic software systems were procedural and logical. The procedural paradigm was epitomized by

Lisp which featured recursive procedures that operated on list structures.

The logical paradigm was epitomized by uniform proof procedure resolution-based derivation (proof) finders. According to the logical paradigm it was "cheating" to incorporate procedural knowledge.

Procedural embedding of knowledge

Planner was invented for the purposes of the procedural embedding of knowledge and was a rejection of the resolution uniform proof procedure paradigm, which

Converted everything to clausal form. Converting all information to clausal form is problematic because it hides the underlying structure of the information.

Then used resolution to attempt to obtain a proof by contradiction by adding the clausal form of the negation of the theorem to be proved. Using only resolution as the rule of inference is problematical because it hides the underlying structure of proofs. Also, using proof by

contradiction is problematical because the axiomatizations of all practical domains of knowledge are inconsistent in practice.

Planner was a kind of hybrid between the procedural and logical paradigms because it combined programmability with logical reasoning. Planner featured a procedural interpretation of logical sentences where an implication of the form (P implies Q) can be procedurally interpreted in the following ways using pattern-directed invocation:

Forward chaining (antecedently):

If assert P, assert Q

If assert not Q, assert not P

Backward chaining (consequently)

If goal Q, goal P

If goal not P, goal not Q

In this respect, the development of Planner was influenced by natural deductive logical systems (especially the one by Frederic Fitch [1952]).

Micro-planner implementation

A subset called Micro-Planner was implemented by Gerry Sussman, Eugene Charniak and Terry Winograd and was used in Winograd's natural-language understanding program SHRDLU, Eugene Charniak's story understanding work, Thorne McCarty's work on legal reasoning, and some other projects. This generated a great deal of excitement in the field of AI. It also generated controversy because it proposed an alternative to the logic approach that had been one of the mainstay paradigms for AI.

At SRI International, Jeff Rulifson, Jan Derksen, and Richard Waldinger developed QA4 which built on the constructs in Planner and introduced a context mechanism to provide modularity for expressions in the database. Earl Sacerdoti and Rene Reboh developed QLISP, an

extension of QA4 embedded in INTERLISP, providing Planner-like reasoning embedded in a procedural language and developed in its rich programming environment. QLISP was used by Richard Waldinger and Karl Levitt for program verification, by Earl Sacerdoti for planning and execution monitoring, by Jean-Claude Latombe for computer-aided design, by Nachum Dershowitz for program synthesis, by Richard Fikes for deductive retrieval, and by Steven Coles for an early expert system that guided use of an econometric model.

Computers were expensive. They had only a single slow processor and their memories were very small by comparison with today. So Planner adopted some efficiency expedients including the following:

Backtracking was adopted to economize on the use of time and storage by working on and storing only one possibility at a time in exploring alternatives.

A unique name assumption was adopted to save space and time by assuming that different names referred to different objects. For example, names like Peking (previous PRC capital name) and Beijing (current PRC capital transliteration) were assumed to refer to different objects.

A closed-world assumption could be implemented by conditionally testing whether an attempt to prove a goal exhaustively failed. Later this capability was given the misleading name "negation as failure" because for a goal G it was possible to say: "if attempting to achieve G exhaustively fails then assert (Not G)."

The genesis of Prolog

Gerry Sussman, Eugene Charniak, Seymour Papert and Terry Winograd visited the University of Edinburgh in 1971, spreading the news about Micro-Planner and SHRDLU and casting doubt on the resolution uniform proof procedure approach that had been the mainstay of the Edinburgh Logicians. At the University of Edinburgh, Bruce Anderson implemented a subset of

Micro-Planner called PICO-PLANNER, and Julian Davies (1973) implemented essentially all of Planner.

According to Donald MacKenzie, Pat Hayes recalled the impact of a visit from Papert to Edinburgh, which had become the "heart of artificial intelligence's Logicland," according to Papert's MIT colleague, Carl Hewitt. Papert eloquently voiced his critique of the resolution approach dominant at Edinburgh "...and at least one person upped sticks and left because of Papert."

The above developments generated tension among the Logicians at Edinburgh. These tensions were exacerbated when the UK Science Research Council commissioned Sir James Lighthill to write a report on the AI research situation in the UK. The resulting report [Lighthill 1973; McCarthy 1973] was highly critical although SHRDLU was favorably mentioned.

Pat Hayes visited Stanford where he learned about Planner. When he returned to Edinburgh, he tried to influence his friend Bob Kowalski to take Planner into account in their joint work on automated theorem proving. "Resolution theorem-proving was demoted from a hot topic to a relic of the misguided past. Bob Kowalski doggedly stuck to his faith in the potential of resolution theorem proving. He carefully studied Planner.". Kowalski [1988] states "I can recall trying to convince Hewitt that Planner was similar to SL-resolution." But Planner was invented for the purposes of the procedural embedding of knowledge and was a rejection of the resolution uniform proof procedure paradigm. Colmerauer and Roussel recalled their reaction to learning about Planner in the following way:

"While attending an IJCAI convention in September '71 with Jean Trudel, we met Robert Kowalski again and heard a lecture by Terry Winograd on natural language processing. The fact that he did not use a unified formalism left us puzzled. It was at this time that we learned of the existence of Carl Hewitt's programming language, Planner. The lack of formalization of this language, our ignorance of Lisp and, above all, the fact that we were absolutely devoted to logic meant that this work had little influence on our later research."

In the fall of 1972, Philippe Roussel implemented a language called Prolog (an abbreviation for PROgrammation en LOGique – French for "programming in logic"). Prolog programs are generically of the following form (which is a special case of the backward-chaining in Planner):

When goal Q, goal P1 and ... and goal Pn

Prolog duplicated the following aspects of Micro-Planner:

Pattern directed invocation of procedures from goals (i.e. backward chaining)

An indexed data base of pattern-directed procedures and ground sentences.

Giving up on the completeness paradigm that had characterized previous work on theorem proving and replacing it with the programming language procedural embedding of knowledge paradigm.

Prolog also duplicated the following capabilities of Micro-Planner which were pragmatically useful for the computers of the era because they saved space and time:

Backtracking control structure

Unique Name Assumption by which different names are assumed to refer to distinct entities, e.g., Peking and Beijing are assumed to be different.

Reification of Failure. The way that Planner established that something was provable was to successfully attempt it as a goal and the way that it establish that something was unprovable was to attempt it as a goal and explicitly fail. Of course the other possibility is that the attempt to prove the goal runs forever and never returns any value. Planner also had a (not expression) construct which succeeded if expression failed, which gave rise to the "Negation as Failure" terminology in Planner.

Use of the Unique Name Assumption and Negation as Failure became more questionable when attention turned to Open Systems.

The following capabilities of Micro-Planner were omitted from Prolog:

Pattern-directed invocation of procedural plans from assertions (i.e., forward chaining)

Logical negation, e.g., (not (human Socrates)).

Prolog did not include negation in part because it raises implementation issues. Consider for example if negation were included in the following Prolog program:

```
not Q.
```

```
Q :- P.
```

The above program would be unable to prove not P even though it follows by the rules of mathematical logic. This is an illustration of the fact that Prolog (like Planner) is intended to be a programming language and so does not (by itself) prove many of the logical consequences that follow from a declarative reading of its programs.

The work on Prolog was valuable in that it was much simpler than Planner. However, as the need arose for greater expressive power in the language, Prolog began to include many of the capabilities of Planner that were left out of the original version of Prolog.

References

Bibliography

External links

Alain Colmerauer's and Philippe Roussel's 1992 account of the birth of Prolog at the Wayback Machine (archived July 27, 2003)